

Lab 1. DC 모터 PID 제어

실습 0. F/W 컴파일 및 다운로드 테스트

1. 자신의 작업 폴더를 하나 생성. (이하에서는 자신의 작업 폴더를 “D:\USR” 이라고 한다.)
2. 바탕화면에 “C6701” 명령창 아이콘을 더블 클릭하여 도스 창을 연다. 환경변수가 설정되어야 하므로 반드시 이 명령창을 활용.
3. 과목 홈페이지에서 예제코드(MeTR_DC.zip)를 다운로드 하여 작업 폴더에 저장하고 압축 해제.
4. 명령창에서 현재 폴더의 위치를 예제코드가 저장된 위치로 이동. (cd D:\USR)
5. 명령창에서 키보드를 통해 “make”를 실행하면 컴파일이 진행되고, 에러가 없다면 “.out” 파일 생성.
6. PC의 USB Port와 실습 KIT를 연결하고, 전원을 인가.
7. 명령창에서 키보드를 통해 “d” 또는 “d.bat”를 실행하면 PC의 USB Port를 통하여 프로그램이 다운로드 된 후 실행됨.
8. serial 통신 터미널(baud rate: 115,200bps)에서 UART로 전송된 string을 확인 가능.
9. 이 실습에서 사용된 코드를 출발점으로 다음 실습을 진행.

참고: 위에서 설명한 내용은 RAM에 컴파일한 코드를 다운로드하는 절차. RAM에 다운로드하였으므로 전원을 껐다 켜면 다운로드한 코드는 사라지고, FLASH에 저장되었던 코드가 실행됨. 만약, 자신이 작성한 코드가 전원이 꺼졌다 켜질 때 자동으로 실행되기를 원하면 실행 코드를 FLASH에 fusing하여야 함. 이를 위해서는 위의 5번째 단계에서 “make flash”를 실행하여 컴파일하고, 7번째 단계에서는 “f” 또는 “f.bat”를 실행. DSP 보드의 reset 스위치를 누르는 동작은 전원을 껐다 켜는 것과 동일한 효과가 발생.

실습 1. PWM을 이용한 DC 모터 구동

1. PWM 발생 회로는 내장 FPGA에 Verilog 코딩으로 구현. 관련 레지스터는 다음과 같음.

IO	Word Offset	Addr. Offset	Read / Write	Description
DC_DRV_EN	0x020	0x0200_0080	R/W	PWM Driver En/Dis Control
Bit[31:1]				Bit0
N.A.				En/Dis
En/Dis	0: DC Motor PWM Driver Disable 1: DC Motor PWM Driver Enable			

IO	Word Offset	Addr. Offset	Read / Write	Description
DC_PWM_R	0x021	0x0200_0084	R/W	DC Motor right Duty ratio setting
DC_PWM_L	0x022	0x0200_0088	R/W	DC Motor left Duty ratio setting
Bit[31:12]				Bit[11:0]
N.A.				Duty
Duty	PWM duty ratio setting PWM 주파수는 대략 20 kHz 0x000 : 0% PWM 0x800 : 50% PWM 0xFFFF : 100% PWM			

2. 모터의 기구부는 아직 연결하지 않은 상태에서 홈페이지에 배포한 MeTR_DC.zip 을 실행하여 PWM 이 출력되는지 확인.

I/O보드 J4의 2번, 3번 핀을 오실로스코프로 관찰하면 50% duty의 서로 역상인 PWM 신호 확인.

3. *DC_PWM_R 에 12 비트의 정수값을 넣었을 때, 위 표와 같이 PWM의 duty가 변화됨을 관찰.
4. PWM duty를 설정하는 함수를 다음과 같은 형태로 작성하여 0~100%에 해당하는 PWM 파형이 출력되도록 구현한다.

```
void PWMOut(float dutyratio);
```

함수의 인자인 dutyratio에는 -50.0 ~ 50.0 범위의 값을 인가하면 PWM 파형은 0~100% 듀티비로 발생하도록 작성한다. (dutyratio에 0을 입력하면 50% 듀티 파형이 생성되어 정지함)

일단 이 함수를 생성한 후에는 코드에서 DC_PWM_R 에 직접 값을 쓰지 말고, 이 함수를 통하여 PWM duty를 변경하도록 한다.

출력 파형의 PWM duty는 0 또는 100% duty로 saturation 되어야 한다.

5. PWMOut 함수가 동작하면 모터와 보드를 연결하고 PWMOut 함수를 테스트.

실습 2. 엔코더 신호를 이용한 위치 측정

1. 코드를 실행하여 엔코더 카운터 값의 변화를 확인. *DC_DRV_EN을 0으로 설정하여 PWM 드라이버를 OFF한 상태에서 *DC_ENC_POSCNT_R의 32비트 값을 UART로 대략 0.5초에 1회 출력하도록 코딩. 이 코드를 컴파일 및 실행한 상태에서 바퀴를 손으로 회전하면 Encoder counter 값이 증가/감소, 회전 방향을 바꾸면 감소/증가하는 현상을 관찰할 수 있음. 엔코더 파형은 J4의 4, 5번 핀에서 관찰 가능. 또한, *DC_ENCCNT_CLR에 아무 값이나 쓰면 counter의 값이 0으로 reset 됨.
2. 본 실험에서 사용하는 DC 모터에는 모터 1회전 당 512개의 펄스가 발생하는 magnetic 엔코더가 내장됨. 또한, 바퀴는 DC 모터 회전축에서 7.5:1 비율의 기어를 통하여 연결되므로 바퀴 1회전은 DC 모터 7.5바퀴 회전에 해당. 바퀴 1회전 결과 발생하는 엔코더 카운터 값의 변화를 정량적으로 확인. (정방향, 역방향 모두) 아울러 *DC_ENCCNT_MODE 설정을 변경하면 1체배, 2체배, 4체배를 선택할 수 있는데, 이 설정 변화에 따라 엔코더 카운터 값의 변화가 정량적으로 맞는지 확인.
 - 1체배에서 바퀴 1회전은 엔코더 카운트 값 얼마에 해당하나?
 - 1체배에서 엔코더 카운트 값 1의 변화는 각도 얼마에 해당하나? (분해능)

IO	Word Offset	Addr. Offset	Read / Write	Description
DC_ENCCNT_CLR	0x023	0x0200_008C	W	Encoder counter clear
Bit[31:1]				Bit0
N.A.				ECC
ECC	Encoder counter clear All encoder position counters are cleared to zero by writing any value to bit0.			

IO	Word Offset	Addr. Offset	Read / Write	Description
DC_ENCCNT_MODE	0x024	0x0200_0090	R/W	Encoder counter mode
Bit[31:2]				Bit[1:0]
N.A.				MODE
MODE	Encoder counter mode 2'b00: 1X, 2'b01/2'b10: 2X, 2'b11: 4X			

IO	Word Offset	Addr. Offset	Read / Write	Description
DC_ENCST_R	0x028	0x0200_00A0	R	Right encoder pulse status
DC_ENCST_L	0x029	0x0200_00A4	R	Left encoder pulse status
Bit[31:2]				Bit[1:0]
N.A.				ENCPHA
ENCPHA	ENCPHA[1:0] = {B, A}			

IO	Word Offset	Addr. Offset	Read / Write	Description
DC_ENC_POSCNT_R	0x02A	0x0200_00A8	R	Right Encoder Position Counter
DC_ENC_POSCNT_L	0x02B	0x0200_00AC	R	Left Encoder Position Counter
Bit[31:0]				
EncPos				
EncPos	Encoder Counter Value(32 bit 2's complement value) 32'h8000_0000 ~ 32'h7FFF_FFFF			

3. 바퀴의 회전 각도를 측정하기 위하여 다음 형태의 함수를 작성.

`float GetAngle();`

현재 바퀴의 회전 각도를 **deg** 단위로 출력하도록 위 함수를 코딩.

모터 회전축 1회전에 엔코더 펄스 카운터 값이 512 증가 또는 감소(회전 방향에 따라)하며, 모터와 바퀴 사이에는 7.5:1 비율의 기어가 장착되어 있음.

- 모터 1회전 ⇔ 엔코더 펄스 카운터 값 512 증가 또는 감소
- 바퀴 1회전 ⇔ 모터 7.5회전

4. [선택적]바퀴의 회전 각도 측정에 하드웨어가 제공하는 DC_ENC_POSCNT_R 의 엔코더 카운트 값 대신 DC_ENCST_R 에서 엔코더 펄스 상태를 직접 확인하여 엔코더 카운터를 FW에서 소프트웨어적으로 구현하여 다음 형태의 함수를 작성.

`float GetAngleSW();`

이 함수를 구현하고 호출하는 데는 약간의 제약 조건이 있음.

- 이 함수를 지속적으로 호출하여야 하는데, 시간 지연적인 요소가 없어야 함.
 - main 함수 내에서 무한루프로 위 함수를 호출하여 카운터 값을 증감하고, 그로부터 각도를 연산
- ```
while (1) {
 angleSW = GetAngleSW();
}
```
- 이 무한루프 내에는 `delay`, 또는 `printf`와 같은 시간 지연이 절대 없어야 함.
  - DC\_ENC\_POSCNT\_R 을 사용하지 않고 구한 각도는 `printf` 대신 타이머 인터럽트의 USBMonitor 기능을 활용하여 확인. USBMonitor 사용법은 일반 오실로스코프 사용법과 거의 같으며, 사용법은 별도 문서 참조.
  - GetAngle() 로 구한 각도와 GetAngleSW() 로 구한 각도를 비교하여 유사성을 확인.

### 실습 3. PID 제어기 구현

1. \*PWMDRVEN 값을 1로 설정하여 PWM 드라이버를 다시 ON.
2. Timer ISR에서 PID 제어를 코딩. 현재 타이머 인터럽트 주파수는 1 kHz로 적용됨.
3. Timer ISR 내부에서 GetAngle 함수와 PWMOut 함수를 활용.
4. PID gain은 각각 define으로 설정하여 쉽게 수정할 수 있도록 코딩.
5. position 제어가 제대로 되는지 확인. position 제어는 step response를 확인하며, step response를 바탕으로 PID 제어를 튜닝. 이 때, GetAngle을 이용하여 읽은 현재 각도와 reference 각도, 제어입력 등을 USBMonitor 기능을 활용하여 관측하면 제어공학에서 학습한 오버슈트 등의 파형을 확인 가능. USBMonitor 사용법은 일반 오실로스코프 사용법과 거의 같으며, 사용법은 별도 문서 참조.

### 실습 4. 위치 궤적 트래킹

1. 실습 3에서는 목표 위치를 step 형식으로 적용. 반면, 사다리꼴 속도 프로파일에 대응하는 위치 궤적을 사용하여 부드러운 동작 구현 가능. 이 위치 궤적은 가속, 등속, 그리고 감속 구간으로 구성되며, 가/감속 구간의 각가속도, 등속 구간의 등각속도, 그리고 이동할 각도 등 3개의 파라미터를 조정하여 궤적의 특성을 변경. 이 때, 경우에 따라 등속구간이 나타나지 않는 경우에도 대응하여야 함에 주의.
2. 위치 레퍼런스를 생성하는 함수를 생성하고, 이 함수를 main 함수 또는 Timer ISR에서 호출.
3. USBMonitor로 각도 레퍼런스, 측정된 각도, 위치 오차, 그리고 제어 입력 등을 확인.
4. Feedforward 제어를 적용하면 feedback 제어기의 이득을 높이지 않고도 트래킹 오차를 줄이는데 매우 효과적. Feedforward 제어기도 적용하여 적용 전후의 추종 오차를 비교.